

Multi-Head *Attention* in Transformers

A comprehensive deep dive into the mechanism that allows transformer models to jointly attend to information from different representation subspaces — the engine behind modern AI.

● Deep Learning

Attention Mechanism

NLP / Vision

Uditya Narayan Tiwari

// Technical Author · AI Researcher

🌐 Portfolio

🐙 GitHub

🌐 LinkedIn

📖 Knowledge Base

// TABLE OF CONTENTS

01 Overview & Motivation

03 Multi-Head Architecture

05 What Each Head Learns

07 Variants & Comparisons

02 Scaled Dot-Product Attention

04 Step-by-Step Computation

06 Parameters & Complexity

08 Applications

Overview & Motivation

The Transformer architecture, introduced in *"Attention Is All You Need"* (Vaswani et al., 2017), revolutionized sequence modeling by replacing recurrence and convolution entirely with attention mechanisms. At its core lies **Multi-Head Attention (MHA)** — the mechanism that enables models to simultaneously attend to information from multiple representational subspaces.

Unlike recurrent models that process tokens sequentially, or convolutional models limited by fixed receptive fields, multi-head attention captures both **local and global dependencies** in a single operation, making it both powerful and parallelizable.

"Instead of performing a single attention function with d_{model} -dimensional keys, values and queries, we found it beneficial to linearly project the queries, keys and values h times..." — Vaswani et al., 2017



Core Insight: A single attention head can only focus on one type of relationship at a time. Multiple heads running in parallel allow the model to simultaneously capture syntactic structure, semantic similarity, coreference, positional patterns, and more.

Scaled Dot-Product Attention

Before understanding multi-head attention, we must understand the building block: **Scaled Dot-Product Attention**. It operates on three matrices derived from the input — **Query (Q)**, **Key (K)**, and **Value (V)**.

```
Attention(Q, K, V) = softmax( (Q · KT) / √dk ) · V
```

Where:

$Q \in \mathbb{R}^{(n \times d_k)}$ — Query matrix

$K \in \mathbb{R}^{(m \times d_k)}$ — Key matrix

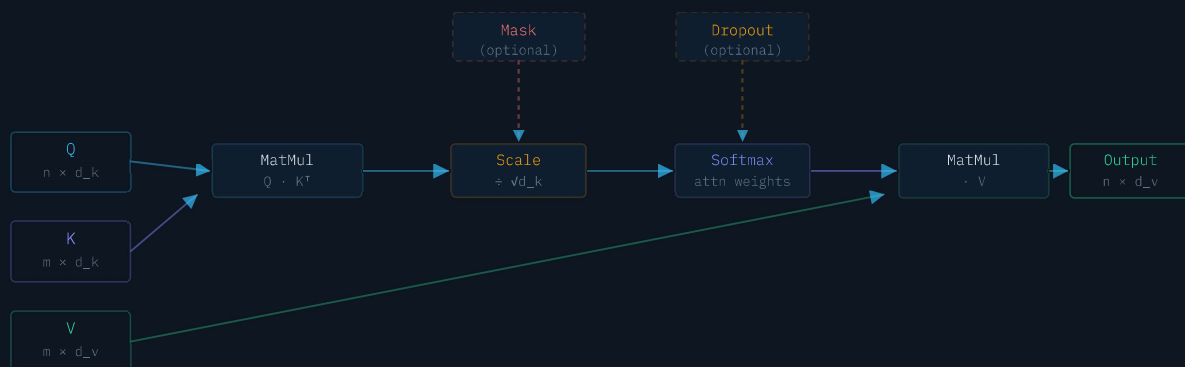
$V \in \mathbb{R}^{(m \times d_v)}$ — Value matrix

d_k — Dimension of keys (scaling factor)

Why the Scaling Factor?

Without scaling by $\sqrt{d_k}$, the dot products grow large in magnitude as d_k increases. Large values push the softmax into regions with extremely small gradients, which makes training unstable. The scaling ensures attention weights remain in a numerically stable regime regardless of dimension.

```
// SCALED DOT-PRODUCT ATTENTION — FORWARD PASS
```





Masking in Decoders: In autoregressive (decoder-only) models, a **causal mask** is applied before the softmax to prevent positions from attending to future tokens — preserving the left-to-right generation property.

// 03

The Multi-Head Architecture

Multi-Head Attention runs **h parallel attention heads**, each operating on *learned projections* of the original Q, K, V matrices. The results are concatenated and projected once more through a final weight matrix.

MULTI-HEAD ATTENTION FORMULA

$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h) \cdot W^O$

where each head is computed as:

$\text{head}_i = \text{Attention}(Q \cdot W_i^Q, K \cdot W_i^K, V \cdot W_i^V)$

Projection matrices:

$W_i^Q \in \mathbb{R}^{(d_{\text{model}} \times d_k)}$ — Query projection for head i

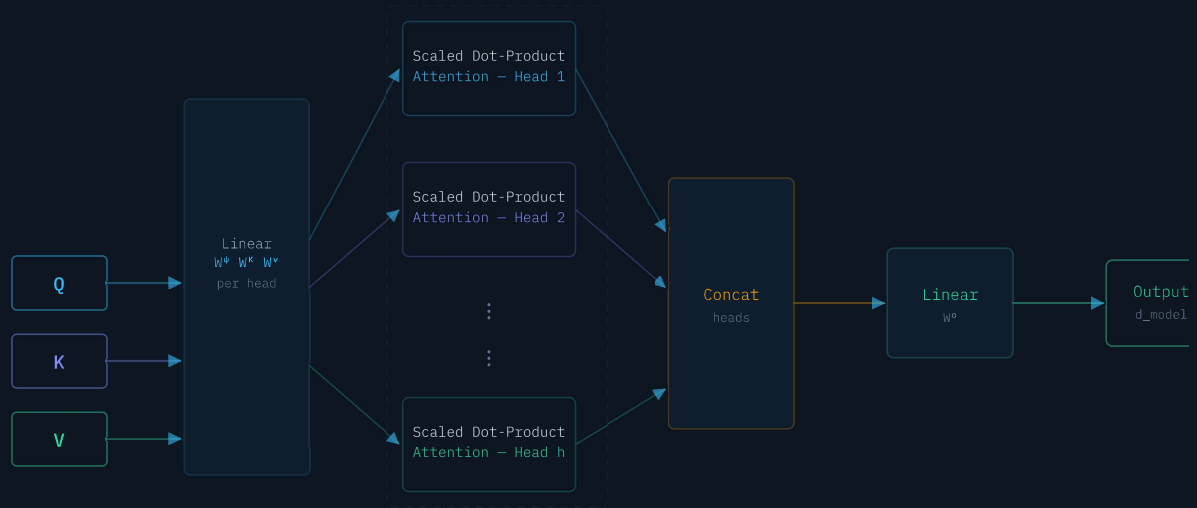
$W_i^K \in \mathbb{R}^{(d_{\text{model}} \times d_k)}$ — Key projection for head i

$W_i^V \in \mathbb{R}^{(d_{\text{model}} \times d_v)}$ — Value projection for head i

$W^O \in \mathbb{R}^{(h \cdot d_v \times d_{\text{model}})}$ — Output projection

Typical: $d_k = d_v = d_{\text{model}} / h$

```
// MULTI-HEAD ATTENTION — ARCHITECTURE DIAGRAM
```



h attention heads run in parallel



```
// 04
```

Step-by-Step Computation

Given input $X \in \mathbb{R}^{(\text{seq_len} \times d_model)}$, here is the full computational flow through a Multi-Head Attention layer.

01

Linear Projection into Q, K, V subspaces

For each head i , compute: $Q_i = X \cdot W_i^Q$, $K_i = X \cdot W_i^K$, $V_i = X \cdot W_i^V$. These projections transform the input into head-specific representation subspaces of dimension $d_k = d_{\text{model}} / h$.

02

Compute Raw Attention Scores

For each head, compute raw scores: $S_i = Q_i \cdot K_i^T \in \mathbb{R}^{(n \times m)}$. Each entry $S[i][j]$ measures how much token i should attend to token j — a raw compatibility score between query and key vectors.

03

Scale the Scores

Divide by $\sqrt{d_k}$ to normalize variance. As d_k grows, dot products can explode in magnitude — scaling keeps softmax in its informative, gradient-friendly range rather than saturating near 0 or 1.

04

Apply Mask (Decoder Only)

In decoder self-attention, set future positions to $-\infty$ before softmax. After exponentiation, these become 0 — effectively masking any information leakage from future tokens during autoregressive generation.

05

Softmax → Attention Weights

Apply row-wise softmax: $A_i = \text{softmax}(S_i / \sqrt{d_k})$. Each row sums to 1, giving a probability distribution over positions — telling each query token how much weight to assign to each key-value pair.

06

Weighted Sum over Values

Compute head output: $\text{head}_i = A_i \cdot V_i \in \mathbb{R}^{(n \times d_v)}$. Each output position is a weighted blend of value vectors, with weights determined by attention scores — creating a context-aware representation.

07

Concatenate All Heads

Stack outputs along the feature dimension: $\text{Concat}(\text{head}_1, \dots, \text{head}_h) \in \mathbb{R}^{(n \times (h \cdot d_v))}$. Each head has independently captured different patterns — concatenation preserves

all of them for the final projection step.

08

Final Output Projection

Multiply by output weight matrix: $\text{Output} = \text{Concat}(\text{heads}) \cdot W^o \in \mathbb{R}^{(n \times d_{\text{model}})}$. This learned projection blends information from all heads and restores the original model dimensionality for downstream processing.

// 05

What Each Head Learns

Research analyzing trained transformer models reveals that different attention heads specialize in distinct linguistic and structural patterns. GPT-2 and BERT studies (Clark et al. 2019, Vig & Belinkov 2019) show remarkable specialization.

h_1

Positional Adjacency

Attends to immediately adjacent tokens — learning local n-gram patterns and surface-level syntax.

h_2

Coreference Resolution

Links pronouns back to their antecedents — "it" → "the car" across sentence boundaries.

h_3

Syntactic Dependency

Captures subject-verb-object relationships and parse tree structure across longer spans.

h_4

Semantic Similarity

Groups semantically related tokens regardless of position — "bank" near financial terms vs. river terms.

h_5

Long-Range Dependency

Attends to distant tokens that are logically or semantically connected, overcoming recurrence limitations.

h_6

Named Entity Focus

Specializes in entity-related attention — proper nouns, dates, numerical expressions and their attributes.

h_7

h_8

Broad Context

Distributes attention widely — approximating a "bag of words" summary of the global context.

Task-Specific

Learns patterns specific to the fine-tuning task — sentiment cues, question-answer links, etc.



Head Pruning Research: Michel et al. (2019) found that in BERT, a significant fraction of heads can be pruned at test time with minimal performance loss. This suggests only a subset of heads are critical for any given task — pointing toward head-level sparsity as an efficiency avenue.

// 06

Parameters & Complexity

TOTAL PARAMETERS

$$4 \cdot d^2$$

4 weight matrices $\times d_{\text{model}}^2$

TIME COMPLEXITY

$$O(n^2 \cdot d)$$

Quadratic in sequence length

SPACE COMPLEXITY

$$O(n^2 \cdot h)$$

Attention maps per head

GPT-3 CONFIG

96 heads

$d_{\text{model}}=12288$, $d_k=128$

PARAMETER COUNT BREAKDOWN

For each head i :

$W_i^\Psi: d_{\text{model}} \times d_k$ params

$W_i^K: d_{\text{model}} \times d_k$ params

$W_i^v: d_{\text{model}} \times d_v$ params

Summed over h heads ($d_k = d_v = d_{\text{model}}/h$):

$$3 \times h \times d_{\text{model}} \times (d_{\text{model}}/h) = 3 \times d_{\text{model}}^2$$

Output projection:

$$W^o: h \times d_v \times d_{\text{model}} = d_{\text{model}}^2$$

Total: $4 \times d_{\text{model}}^2$ parameters

(independent of number of heads h !)



Key Insight: Increasing the number of heads does NOT increase parameter count — it redistributes the same budget into more specialized, lower-dimensional subspaces. More heads = more diverse patterns, same cost.

// 07

Variants & Comparisons

The original Multi-Head Attention has inspired numerous architectural variants designed to address the $O(n^2)$ bottleneck, reduce memory footprint, or improve performance on specific tasks.

VARIANT	CORE IDEA	COMPLEXITY	USED IN	FULL ATTN?
Multi-Head Attention (MHA)	Separate KV heads per query head	$O(n^2 \cdot d)$	BERT, GPT-2	✓ yes

VARIANT	CORE IDEA	COMPLEXITY	USED IN	FULL ATTN?
Multi-Query Attention (MQA)	Single shared K and V across all heads	$O(n^2 \cdot d)$	PaLM, Falcon	~ partial
Grouped-Query Attention (GQA)	K/V shared within groups of heads	$O(n^2 \cdot d)$	LLaMA 2/3, Mistral	~ partial
Sparse Attention	Attend only to subset of positions	$O(n \cdot \sqrt{n} \cdot d)$	GPT-3, Longformer	X no
Linear Attention	Kernel approximation of softmax	$O(n \cdot d^2)$	Performer, FNet	X approx
Flash Attention	IO-aware tiling — same result, faster	$O(n^2 \cdot d)$	GPT-4, Llama 3	✓ yes
Cross-Attention	Q from decoder, K/V from encoder	$O(n \cdot m \cdot d)$	T5, Whisper, DALL-E	✓ yes

Three Attention Modes in Transformers



Encoder Self-Attention: All tokens attend to all tokens bidirectionally. Used in BERT-style models for tasks requiring full context (classification, NER, QA over a passage).



Masked (Causal) Self-Attention: Tokens attend only to past tokens. Used in GPT-style decoder models for autoregressive generation — each token is generated conditioned only on what came before.



Cross-Attention (Encoder-Decoder): Q from the decoder attends to K, V from the encoder output. Used in seq2seq models like T5 and Whisper to condition generation on an external context.

// 08

Real-World Applications

Multi-Head Attention is the computational backbone of virtually every state-of-the-art AI system across modalities.

LANGUAGE MODELS

GPT / BERT / T5

Text generation, classification, translation, summarization

VISION

ViT / DINO

Image patches as tokens — classification, detection, segmentation

MULTIMODAL

CLIP / DALL-E

Cross-modal attention between text and image representations

CODE

Codex / Claude

Long-range attention critical for function definitions and call sites

BIOLOGY

AlphaFold 2

Protein structure prediction via pairwise attention between residues

AUDIO

Whisper / MusicLM

Speech recognition and audio generation via spectrogram transformers



Further Resources

All learning materials, notes, and implementations related to this report are available at:

<https://udityaknowledgebase.netlify.app/>

Key references: Vaswani et al. (2017) "Attention Is All You Need" · Clark et al. (2019) "What Does BERT Look at?" · Michel et al. (2019) "Are Sixteen Heads Really Better than One?" · Dao et al. (2022) "FlashAttention"

Uditya Narayan Tiwari

// AI Researcher · Technical Writer

Multi-Head Attention in Transformers

Technical Report · 2024

→ Portfolio

→ GitHub

→ LinkedIn

→ Knowledge Base